Software Engineering 1: Introduction to Software Security

COMP23311

Week 10, Session 1



Pre-cap

- We will define **standard notions of security** and use them to evaluate a system's **confidentiality, integrity,** and **availability**

Pre-cap

- We will define **standard notions of security** and use them to evaluate a system's **confidentiality, integrity,** and **availability**

- Explain standard **software security problems** in real-world applications and examples

Pre-cap

- We will define **standard notions of security** and use them to evaluate a system's **confidentiality, integrity,** and **availability**
- Explain standard **software security problems** in real-world applications and examples
- Discuss **common vulnerabilities** found in software

Motivating Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int getPassword() {
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

int main(){
  printf(”Enter your password:\n");
  int x = getPassword();
  if(x){
     printf("Access Denied\n");
     exit(0);
  }
  printf("Access Granted\n");
  return 0;
}
```

Motivating Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int getPassword() {
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

int main(){
  printf(”Enter your password:\n");
  int x = getPassword();
  if(x){
     printf("Access Denied\n");
     exit(0);
  }
  printf("Access Granted\n");
  return 0;
}
```

Motivating Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int getPassword() {
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

int main(){
  printf(”Enter your password:\n");
  int x = getPassword();
  if(x){
     printf("Access Denied\n");
     exit(0);
  }
  printf("Access Granted\n");
  return 0;
}
```

- What happens if the user enters “SMT"?

Motivating Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int getPassword() {
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

int main(){
  printf(”Enter your password:\n");
  int x = getPassword();
  if(x){
     printf("Access Denied\n");
     exit(0);
  }
  printf("Access Granted\n");
  return 0;
}
```

- What happens if the user enters “SMT"?
- On a Linux x64 platform running GCC 4.8.2, an input consisting of 24 arbitrary characters followed by *]*, <*ctrl-f*>, and @, will bypass the “Access Denied” message 

Motivating Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int getPassword() {
  char buf[4];
  gets(buf);
  return strcmp(buf, "SMT");
}

int main(){
  printf(”Enter your password:\n");
  int x = getPassword();
  if(x){
     printf("Access Denied\n");
     exit(0);
  }
  printf("Access Granted\n");
  return 0;
}
```

- What happens if the user enters “SMT"?
- On a Linux x64 platform running GCC 4.8.2, an input consisting of 24 arbitrary characters followed by *]*, <*ctrl-f*>, and @, will bypass the “Access Denied” message 
- A more extended input will run over into other parts of the **computer memory** (gets is unsafe: no bounds check)

Motivating Example

- …but I’m using python! 

Motivating Example

- …but I’m using python! 
- Who cares about C anyway?
  - This is not only a C problem… (there are many other “unsafe” languages)

Motivating Example

- …but I’m using python! 
- Who cares about C anyway?
  - This is not only a C problem… (there are many other “unsafe” languages)
- Python itself (interpreter) is written in C!

Motivating Example

- …but I’m using python! 
- Who cares about C anyway?
  - This is not only a C problem… (there are many other “unsafe” languages)
- Python itself (interpreter) is written in C!
- And most existing critical systems (OS, DB, Network prot, etc)

What is Security?

- Security = Protection against **intentional** damage

What is Security?

- Security = Protection against **intentional** damage
- Intentional damage: usually external threats

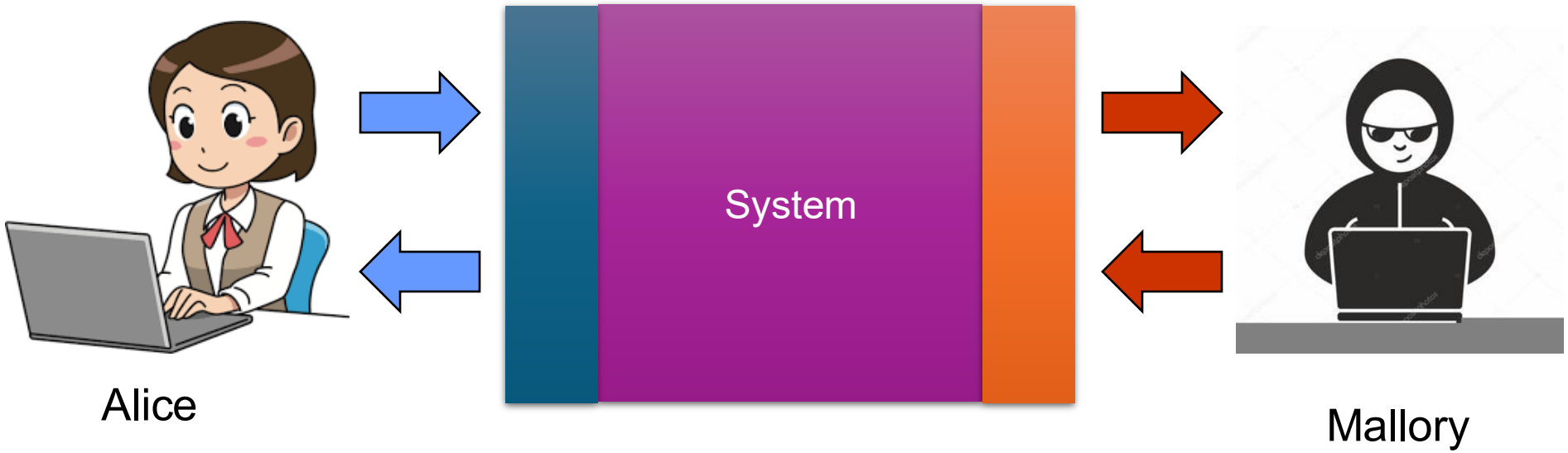
What is Security?

- Security = Protection against **intentional** damage
- Intentional damage: usually external threats
- More features = higher attack risk

What is Security?

- Security = Protection against **intentional** damage
- Intentional damage: usually external threats
- More features = higher attack risk
- Threat Model: assumptions, actors, who can attack, etc.

Threat Model



- Alice: Honest user
- Mallory: Malicious attacker
- System: target of the attacker

Boneh, D. and Mitchell, J., *"Computer and Network Security"*, 2009.

CIA Principle: Expectations of Secure Systems

Confidentiality: Attacker does not learn the user's secrets.

Integrity: Attacker does not undetectably corrupt system's function for the user

Availability: Attacker does not keep system from being useful to the user



CIA Principle: Expectations of Secure Systems

Confidentiality: Attacker does not learn the user's secrets.

Integrity: Attacker does not undetectably corrupt system's function for the user

Availability: Attacker does not keep system from being useful to the user



A system is **secure** if it satisfies the specified/implied security objective, i.e. requirements on confidentiality, integrity, availability

Example: Social Network Application



Linkify

Example: Linkify: fictitious social network

Confidentiality:

Integrity:

Availability:

Example: Social Network Application



Linkify

Example: Linkify: fictitious social network

Confidentiality: Pictures posted by a user can only be seen by that user's friends (3rd-order contacts)

Integrity:

Availability:

Example: Social Network Application



Linkify

Example: Linkify: fictitious social network

Confidentiality: Pictures posted by a user can only be seen by that user's friends (3rd-order contacts)

Integrity: A user can like any given post at most once

Availability:

Example: Social Network Application



Linkify

Example: Linkify: fictitious social network

Confidentiality: Pictures posted by a user can only be seen by that user's friends (3rd-order contacts)

Integrity: A user can like any given post at most once

Availability: The service is operational more than 99.9% of the time on average

Security Failure and Vulnerability

- **Security failure:** a scenario where the software system does not achieve its **security objective**

Security Failure and Vulnerability

- **Security failure:** a scenario where the software system does not achieve its **security objective**

- **Vulnerability:** the **underlying cause** of a security failure

Security Failure and Vulnerability

- **Security failure:** a scenario where the software system does not achieve its **security objective**

- **Vulnerability:** the **underlying cause** of a security failure

- Most software: **no precise, explicit security objectives**
  - Security objectives are not absolute
  - Often traded off against **performance** or **usability**

Security Failure and Vulnerability

- **Security failure:** a scenario where the software system does not achieve its **security objective**

- **Vulnerability:** the **underlying cause** of a security failure

- Most software: **no precise, explicit security objectives**
  - Security objectives are not absolute
  - Often traded off against **performance** or **usability**

- **Implementation bugs** can lead to a substantial **disruption** in the behaviour of the software

Why are there security vulnerabilities?

- **Software** is one of the sources of **security problems**
  - Why do programmers write insecure code?
    - Awareness is the main issue
- Other factors
  - Limited number of **courses** in computer security
  - **Programming** textbooks do not emphasize **security**
  - Limited number of **security audits**
  - Programmers are focused on **implementing features**
  - Security is **expensive** and takes time
  - **Legacy software** (e.g., C is an unsafe language)

Implementation Vulnerability

- **Implementation vulnerability** (or security bug): is bug that
  - makes it possible for an attacker to violate a **security objective**
  - describes vulnerabilities in widely-used software components

Implementation Vulnerability

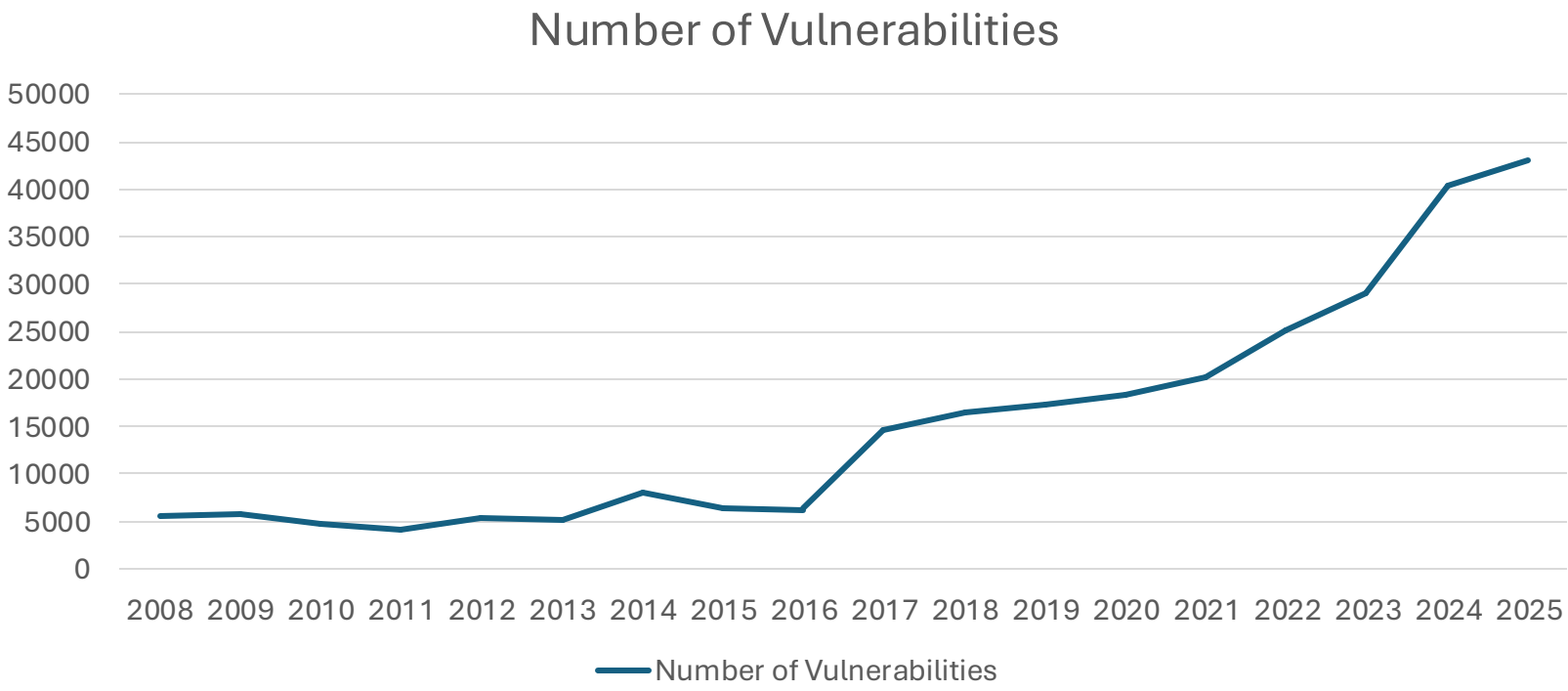
- **Implementation vulnerability** (or security bug): is bug that
  - makes it possible for an attacker to violate a **security objective**
  - describes vulnerabilities in widely-used software components

- **Common Vulnerabilities and Exposures** (CVE) is a publicly available list of entries
  - describes vulnerabilities in widely-used software components
  - it lists ~**300,000** security vulnerabilities

https://www.cve.org/

Common Vulnerabilities and Exposures by year

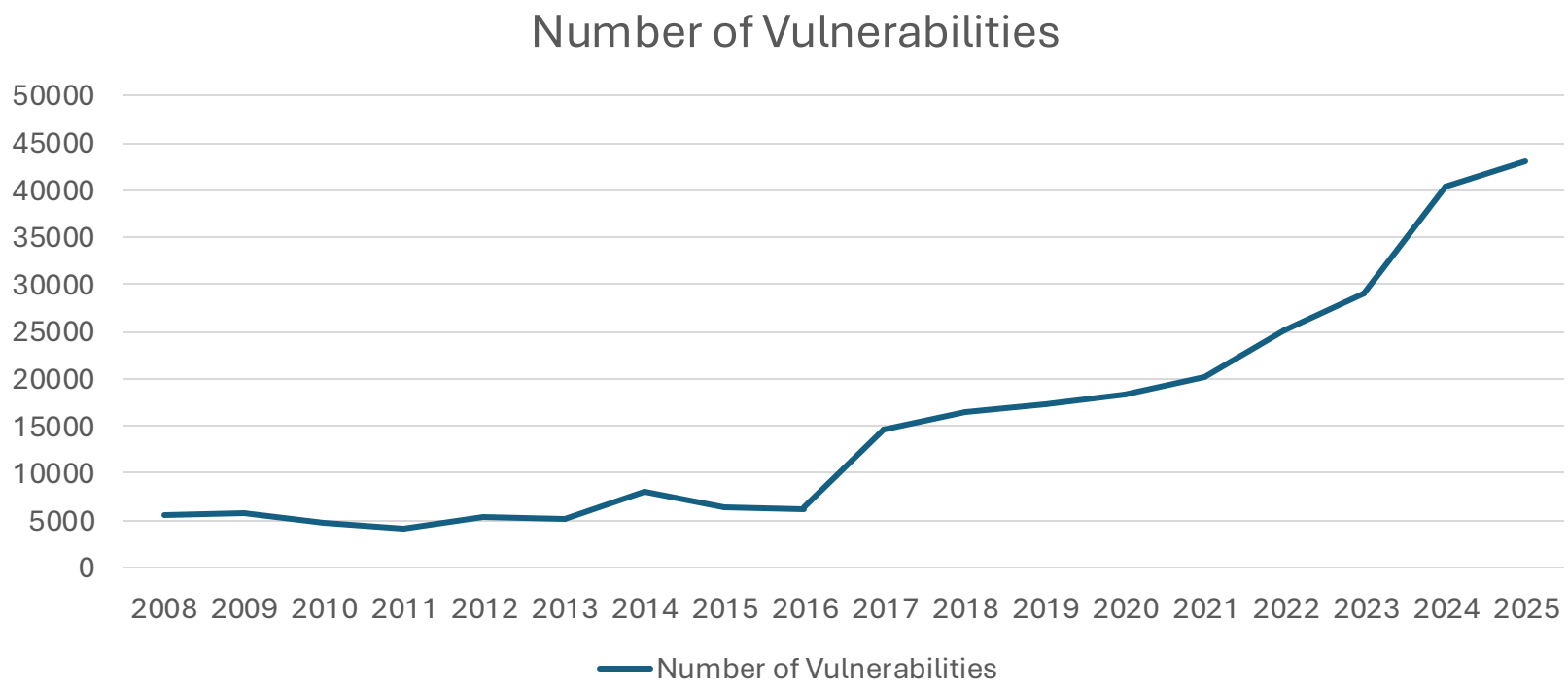
- The number of software vulnerabilities is increasing rapidly
- More than **40,000** vulnerabilities in 2025



https://www.cvedetails.com/

Common Vulnerabilities and Exposures by year

- The number of software vulnerabilities is increasing rapidly
- More than **40,000** vulnerabilities in 2025



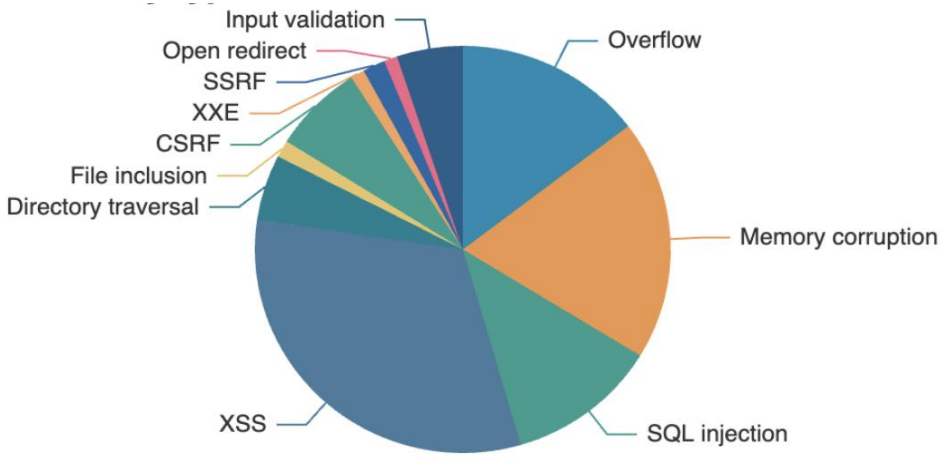
https://www.cvedetails.com/

Most Common Vulnerabilities (since 2015)

By type/catgory

1. XSS (Cross-site scripting)
2. Memory Corruption
4. SQL Injection

...



By impact type

1. Denial of Service

...

Vulnerabilities by impact types

Year	Code Execution	Bypass	Privilege Escalation	Denial of Service	Information Leak
2015	1430	0	79	1793	43
2016	1239	1	149	2050	102
2017	1870	842	1011	3372	1382
2018	1728	643	827	2207	1396
2019	1556	656	900	1697	1313
2020	1691	795	1366	1677	1089
2021	2087	774	1087	2297	911
2022	2067	822	1406	2437	1112
2023	2580	862	1327	2580	1428
2024	3966	647	1077	2469	941
2025	2670	562	925	2323	654
Total	22884	6604	10154	24882	10371

https://www.cvedetails.com/

Memory Corruption:
Null Pointer Dereference

```
int main() {
  double *p = NULL;
  int n = 8;
   for (int i = 0; i < n; ++i) {
     *(p+i) = i*2;
   }
   return 0;
}
```

Scope	Impact
Availability	Crash, exit and restart
Integrity Confidentiality	Execute Unauthorized Code or Commands

Occurs when the application dereferences a pointer that it expects to be valid, but is NULL

Memory Corruption: Double Free

```
int main(){
  char* ptr = (char *)malloc(sizeof(char));
  if(ptr == NULL) return -1;
  *ptr = 'a';
  free(ptr);
  free(ptr);
  return 0;
}
```

Scope	Impact
Availability	Could Crash, exit and restart; could also run silently corrupt heap
Integrity Confidentiality	Execute Unauthorized Code or Commands

The code calls *free()* twice on the same memory address, leading to modification of unexpected memory locations

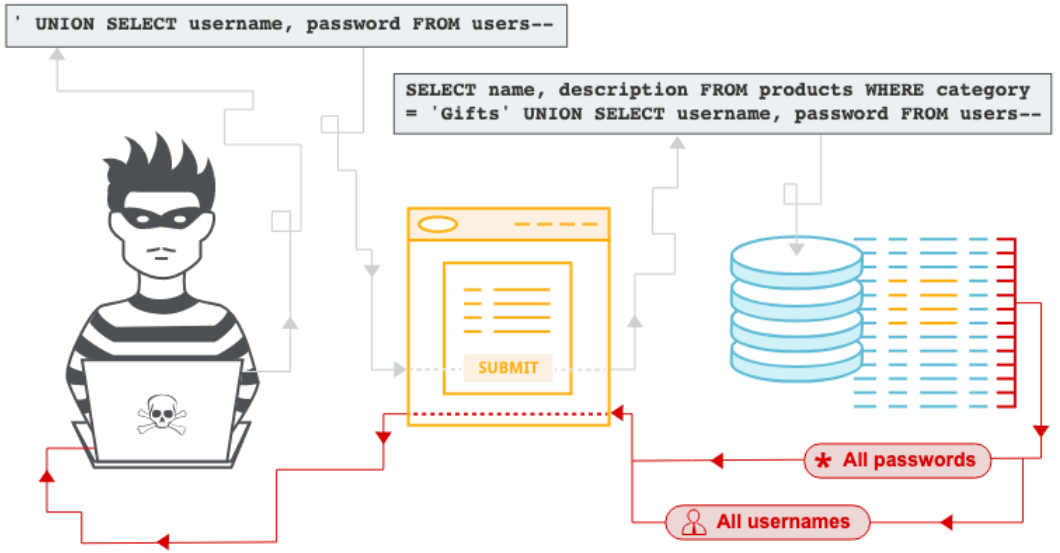
Memory Corruption:
Other types

- Null pointer dereference
- Double free
- Unchecked Return Value to NULL
- Division by zero
- Missing free
- Use after free
- …

SQL Injection

SQL injection allows an attacker to **interfere with the queries to the database** to retrieve **data**

- retrieving hidden data
- subverting application logic
- UNION attacks
- examining the database
- blind SQL injection



https://portswigger.net/web-security/sql-injection

SQL Injection: Example

- A programmer can construct a SQL query with the intention to check **name** and **password** as

```
query = "select * from users where name='" + name + "'"
and pw = '" + password + "'"
```

- However, if the name string is provided by an attacker, the attacker can set name to "John' --"
  - this would remove the password check from the query! (note that -- starts a comment in SQL)

Cross-site Scripting (XSS) Attack

- XSS attacks represent **injection of malicious scripts into trusted websites**

```
<% String eid = request.getParameter("eid"); %>
...
Employee ID: <%= eid %>
```

- XSS allows attackers to **bypass access controls**
  - If *eid* has a value that includes source code, then the code will be executed by the web browser
  - use e-mail or social engineering tricks to lead victims to visit a link to another URL

More xss attacks

Denial of Service (DoS) Attack

- A DoS attack makes a machine or network resource unavailable to its intended users
  - **Flood attacks** occur when the system receives too much traffic for the server to buffer, causing them to slow down
  - **Crashing attacks** exploit vulnerabilities that cause the target system or service to crash

Supply Chain Attacks

- Uses third-party tools or services (“supply chain”) to gain access to a target system or network.

How is a supply chain attack carried out?

- Attacker gain access to the third-party system they plan to exploit ("Upstream" attack)
  - Using stolen credentials, targeting vendors with temporary access to an organization's system, or exploiting an unknown software vulnerability, among other methods
- Once the Upstream attack is done, the "Downstream" attack is performed when that malicious code executes on end-user devices, e.g., via a routine software update.

Types of Supply Chain “Upstream” Attacks

- Browser-based attacks
  - Run malicious code on end-user browsers.
- Software attacks
  - Disguise malware in software updates.
- Open-source attacks
  - Exploit vulnerabilities in open-source code.
- Etc…

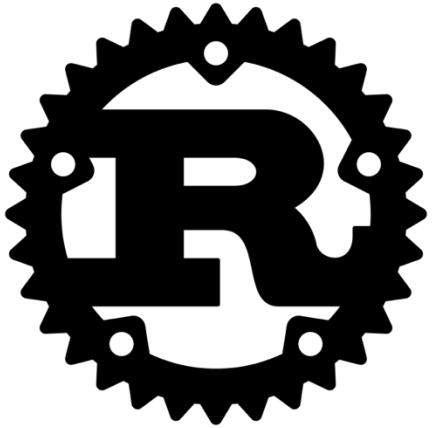
How to prevent Supply Chain Attacks

- Code audit
- Choose third-party software carefully
- Etc

Real World Examples

- SolarWinds attack in 2018
  - Used to transmit malware to clients of SolarWinds software
  - Detailed discussion
- Linux XZ vulnerability in March 2024
  - allowed attacker to execute arbitrary commands through ssh)
  - Detailed discussion

The Rust Programming Language: Memory Safety without sacrificing performance





- Eliminates an entire class of memory-safety vulnerabilities
- Performance comparable to C/C++ (no garbage collection)
- Strong type system reduces logic & API misuse errors (a bit tedious to write)

Any 2025 programmer may want to have Rust on their stack!

Take-Aways

- Security is a fundamental part of software engineering
  - System security = CIA: Confidentiality, Integrity, Availability

- Security failure
  - Scenario where the software system doesn't meet **security objectives**
  - Security Vulnerability**:**

- Different Types of Security Vulnerabilities
  - Memory Corruption
  - XSS attacks
  - SQL Injections
  - DoS
  - Supply chain attacks